Step 1 — Install spaCy and Download Model

```
# Install spaCy
!pip install spacy

# Download English language model
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spac
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/d
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from sp
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from reque
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from type
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from ji
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdow
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 92.8 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Step 2 — Import Libraries and Load Model

```
import spacy
import pandas as pd
from spacy import displacy

# Load spaCy English model
```

```
nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully")
```

```
spaCy model loaded successfully
```

Step 3 — Prepare Dataset (15 Sentences)

```
sentences = [
    "The cat chased the mouse.",
    "She reads a book every day.",
    "The teacher explained the lesson clearly.",
    "The boy kicked the ball and ran away.",
    "I went to the store because I needed milk.",
    "The scientist who discovered the vaccine won an award.",
    "Did you complete the assignment?",
    "Why are students studying late tonight?",
    "The project was completed by the team.",
    "The cake was baked by my mother.",
    "Technology improves our daily lives.",
    "Social media connects people around the world.",
    "The programmer fixed the bug quickly.",
    "Artificial intelligence is transforming industries.",
    "The laptop on the table belongs to John."
]
```

Step 4 — Process Sentences Using spaCy

This step performs tokenization, POS tagging, and dependency parsing.

```
docs = []

for sentence in sentences:
    doc = nlp(sentence)
    docs.append(doc)

    print("\nSentence:", sentence)
    for token in doc:
        print(token.text, token.pos_, token.dep_)
```

```
the DET det
```

```
the DET det
world NOUN pobj
. PUNCT punct

Sentence: The programmer fixed the bug quickly.
The DET det
programmer NOUN nsubj
fixed VERB ROOT
the DET det
bug NOUN dobj
quickly ADV advmod
. PUNCT punct

Sentence: Artificial intelligence is transforming industries.
Artificial ADJ amod
intelligence NOUN nsubj
is AUX aux
transforming VERB ROOT
industries NOUN dobj
. PUNCT punct

Sentence: The laptop on the table belongs to John.
The DET det
laptop NOUN nsubj
on ADP prep
the DET det
table NOUN pobj
belongs VERB ROOT
to ADP prep
John PROPN pobj
. PUNCT punct
```

Step 5 — Extract Dependency Relations

This will create a table showing token, lemma, POS tag, dependency, and head word.

```
data = []

for doc in docs:
    for token in doc:
        data.append([
            doc.text,
            token.text,
            token.lemma_,
            token.pos_,
            token.dep_,
            token.head.text
        ])

df = pd.DataFrame(data, columns=[
    "Sentence",
    "Token",
    "Lemma",
    "POS",
    "Dependency",
    "Head Word"
])

df
```

	Sentence	Token	Lemma	POS	Dependency	Head Word
0	The cat chased the mouse.	The	the	DET	det	cat
1	The cat chased the mouse.	cat	cat	NOUN	nsubj	chased
2	The cat chased the mouse.	chased	chase	VERB	ROOT	chased
3	The cat chased the mouse.	the	the	DET	det	mouse
4	The cat chased the mouse.	mouse	mouse	NOUN	dobj	chased
...	...	...	...	...	...	...
109	The laptop on the table belongs to John.	table	table	NOUN	pobj	on
110	The laptop on the table belongs to John.	belongs	belong	VERB	ROOT	belongs
111	The laptop on the table belongs to John.	to	to	ADP	prep	belongs
112	The laptop on the table belongs to John.	John	John	PROPN	pobj	to
113	The laptop on the table belongs to John.	.	.	PUNCT	punct	belongs

114 rows × 6 columns

Next steps: Generate code with df New interactive sheet

Step 6 — Identify Sentence Structure (Subject, Verb, Object)

```
for doc in docs:
    print("\nSentence:", doc.text)

    subject = ""
    verb = ""
    obj = ""

    for token in doc:
        if token.dep_ == "nsubj":
            subject = token.text
        if token.dep_ == "ROOT":
            verb = token.text
        if token.dep_ == "dobj":
            obj = token.text

    print("Subject:", subject)
    print("Main Verb:", verb)
    print("Object:", obj)
```

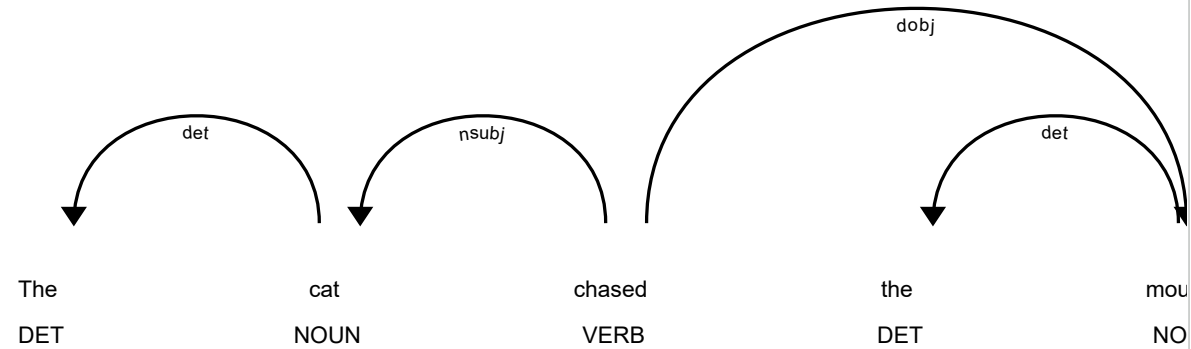
```
Subject: boy
Main Verb: kicked
Object: ball
```

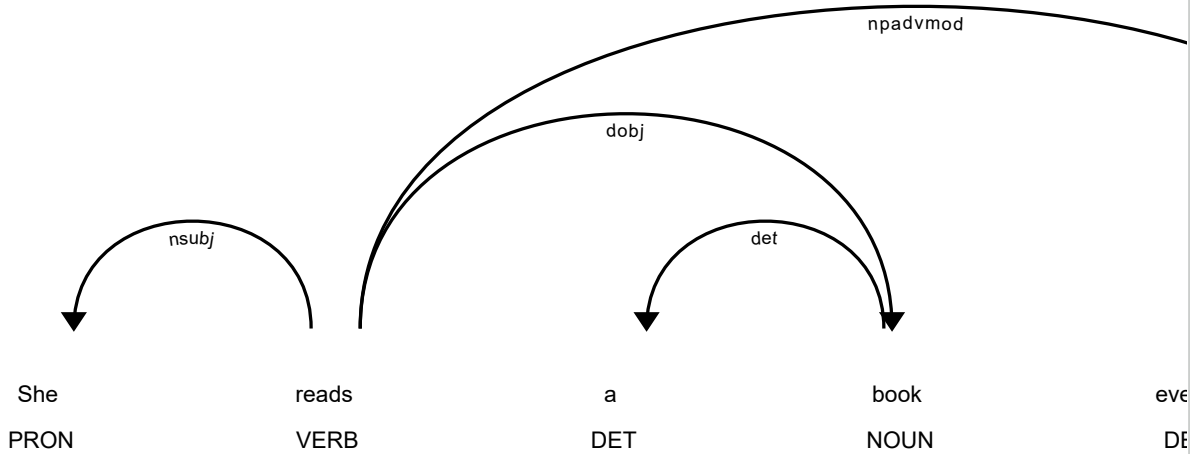
```
Main Verb: are
Object:

Sentence: The project was completed by the team.
Subject:
Main Verb: completed
Object:

Sentence: The cake was baked by my mother.
Subject:
Main Verb: baked
Object:

Sentence: Technology improves our daily lives.
Subject: Technology
Main Verb: improves
Object: lives

Sentence: Social media connects people around the world.
Subject: media
Main Verb: connects
Object: people

Sentence: The programmer fixed the bug quickly.
Subject: programmer
Main Verb: fixed
Object: bug

Sentence: Artificial intelligence is transforming industries.
Subject: intelligence
Main Verb: transforming
Object: industries

Sentence: The laptop on the table belongs to John.
Subject: laptop
Main Verb: belongs
Object:
```

Step 7 — Dependency Tree Visualization

This will display dependency trees for 5 sentences.

```
for i in range(5):
    doc = nlp(sentences[i])
    displacy.render(doc, style="dep", jupyter=True)
```





Step 8 — Show Named Dependency Relations (Optional Check)

```
for doc in docs:
    print("\nSentence:", doc.text)

    for token in doc:
        print(f"{token.text} --> {token.dep_} --> {token.head.text}")
```

```
Sentence: The cat chased the mouse.
The --> det --> cat
cat --> nsubj --> chased
chased --> ROOT --> chased
the --> det --> mouse
mouse --> dobj --> chased
. --> punct --> chased

Sentence: She reads a book every day.
She --> nsubj --> reads
reads --> ROOT --> reads
a --> det --> book
book --> dobj --> reads
every --> det --> day
day --> npadvmod --> reads
. --> punct --> reads
```